

# 07 — Identity & Dependency Specification

*Stable Infrastructure Identity, Relationship Modeling, Graph Semantics & Dependency Safety*

| Field              | Value                                                                                         |
|--------------------|-----------------------------------------------------------------------------------------------|
| Document ID        | CloudShield AI — 07                                                                           |
| Version            | 1.0                                                                                           |
| Status             | Engineering Baseline                                                                          |
| Scope              | Product-wide identity, relationship and dependency semantics                                  |
| Primary Cloud      | AWS for MVP                                                                                   |
| Canonical Identity | CloudShield AI asset identity anchored by deterministic provider identifiers                  |
| Graph Strategy     | Relational dependency model for MVP; graph-optimized storage later if justified               |
| DAIL Position      | Consumes dependency/identity semantics for deterministic change lifecycle and impact analysis |

**Identity answers 'what is this?' Dependency answers 'what does it affect?'**

## Document Contents

1. Purpose and Scope
2. Identity and Dependency Principles
3. Identity Model
4. Tenant, Scope and Identity Boundaries
5. Cloud Resource Identity
6. Asset Identity Lifecycle
7. Identifier Resolution and Reconciliation
8. Relationship Model
9. Dependency Taxonomy
10. Dependency Discovery
11. Dependency Confidence and Provenance
12. Graph Semantics
13. Dependency Queries
14. Change-Centric Dependency Analysis
15. Impact and Blast-Radius Model
16. Invalidation Semantics
17. DAIL Integration
18. Consistency, Freshness and Versioning
19. Concurrency and Reconciliation
20. Security and Authorization
21. Persistence Model
22. MVP Dependency Coverage
23. Example Dependency Scenarios
24. Acceptance Criteria
25. Traceability
26. Glossary

# 1. Purpose and Scope

This specification defines how CloudShield AI identifies infrastructure resources and models relationships among them. Identity provides a stable product representation of cloud resources across observations and time. Dependency modeling captures relationships that matter for security analysis, risk prioritization, remediation planning, impact analysis, verification and trusted-state management.

The specification is product-level. DAIL uses these semantics as an input to deterministic dependency, impact and invalidation controls, but DAIL does not own the complete infrastructure graph.

## 1.1 Scope

- Canonical asset identity.
- Provider identifier normalization.
- Identity reconciliation across observations.
- Relationship and dependency semantics.
- Dependency provenance and confidence.
- Graph traversal and impact queries.
- Change-centric dependency analysis.
- Invalidation of assumptions affected by dependency changes.
- Authorization and tenant isolation for graph data.
- MVP relational implementation baseline.

## 1.2 Non-Goals

- Building a general-purpose CMDB for every possible IT object.
- Inferring arbitrary relationships solely from AI language-model output.
- Treating visualization topology as the authoritative graph.
- Requiring a graph database for the MVP.
- Allowing dependency analysis to authorize infrastructure changes by itself.

# 2. Identity and Dependency Principles

| Principle                 | Required Behavior                                                                                                |
|---------------------------|------------------------------------------------------------------------------------------------------------------|
| Stable identity           | The same logical cloud resource should retain the same product asset identity when deterministically resolvable. |
| Provider traceability     | Provider-native identifiers remain available for reconciliation and execution.                                   |
| No heuristic merging      | Ambiguous identity is surfaced as conflict rather than silently merged.                                          |
| Relationship explicitness | Dependencies are represented as typed relationships, not inferred from UI proximity.                             |
| Evidence-backed graph     | Safety-relevant relationships identify their source/provenance.                                                  |
| Time-aware                | Relationships and identity observations have validity/freshness semantics.                                       |
| Directionality            | Where causality matters, relationships have explicit source and target.                                          |
| Bounded traversal         | Impact queries use explicit depth/scope limits and policy.                                                       |
| Authorization             | Graph queries respect the same tenant and resource authorization boundaries as other data.                       |
| Fail closed               | Insufficient dependency knowledge cannot be presented as complete safety knowledge.                              |

**IDDEP-001:** Identity and dependency results used for consequential decisions shall be deterministic or explicitly marked uncertain.

## 3. Identity Model

CloudShield AI uses multiple identity layers. Provider identity identifies an external object; product asset identity identifies the normalized CloudShield AI object; observation identity identifies a point-in-time capture.

| Identity Layer             | Example                              | Purpose                             |
|----------------------------|--------------------------------------|-------------------------------------|
| Tenant Identity            | tenant_id                            | Product isolation.                  |
| Cloud Scope Identity       | scope_id                             | Account/region/resource boundary.   |
| Asset Identity             | asset_id                             | Stable canonical resource identity. |
| Provider Resource Identity | provider + account + ARN/resource ID | External traceability.              |
| Observation Identity       | observation_id                       | Point-in-time source record.        |
| Snapshot Identity          | snapshot_id                          | Immutable configuration version.    |
| Relationship Identity      | edge_id                              | Stable graph relationship record.   |
| Candidate Identity         | candidate_id + version               | Exact proposed change.              |

### 3.1 Identity Priority

27. Use a provider's globally unique identifier when available.
28. Otherwise use a deterministic composite identity from provider/account/scope/type/resource identifiers.
29. Use stable secondary identifiers only as supporting reconciliation keys.
30. Never use mutable display names as the sole identity.
31. If identity remains ambiguous, create a conflict state rather than an unsafe merge.

## 4. Tenant, Scope and Identity Boundaries

| Boundary      | Identity Rule                                                                  |
|---------------|--------------------------------------------------------------------------------|
| Tenant        | Every asset and relationship belongs to exactly one tenant.                    |
| Cloud account | Provider account is part of resource identity where needed.                    |
| Region        | Region participates in identity where provider resource IDs are region-scoped. |
| Resource type | Canonical type prevents collisions among different resource classes.           |
| Connection    | Connection identifies access path, not the resource itself.                    |
| Scope         | Scope determines visibility/authorization and discovery context.               |

**ID-TEN-001:** The same provider identifier in two different tenant boundaries shall never resolve to the same tenant-owned asset.

**ID-SCOPE-001:** Identity resolution shall operate within an authorized cloud scope.

## 5. Cloud Resource Identity

### 5.1 Canonical Asset Identity

| Field     | Role                              |
|-----------|-----------------------------------|
| asset_id  | Stable CloudShield AI identifier. |
| tenant_id | Ownership boundary.               |
| provider  | Cloud provider.                   |

|                        |                                     |
|------------------------|-------------------------------------|
| account_id             | Provider account.                   |
| region                 | Resource region when applicable.    |
| asset_type             | Canonical resource type.            |
| provider_resource_id   | Provider-native stable ID.          |
| canonical_resource_key | Normalized deterministic key.       |
| display_name           | Human-readable label; not identity. |
| status                 | Current asset lifecycle state.      |

## 5.2 Canonical Resource Key

A canonical resource key should be deterministic, normalized and provider-aware. A conceptual form is:

**provider://account/scope/resource-type/provider-resource-id**

The exact serialization is an implementation detail, but equivalent resources must produce the same key and unrelated resources must not collide.

**ID-ASSET-001:** Canonical resource keys shall be deterministic and collision-resistant within the supported identity domain.

## 6. Asset Identity Lifecycle

| State      | Meaning                                                                  |
|------------|--------------------------------------------------------------------------|
| DISCOVERED | First valid observation establishes a product asset.                     |
| ACTIVE     | Asset is currently observed and managed.                                 |
| STALE      | Expected observation freshness exceeded.                                 |
| MISSING    | Asset is no longer observed in expected scope.                           |
| RETIRED    | Asset has been intentionally/authoritatively removed from managed scope. |
| CONFLICTED | Identity resolution cannot safely determine equivalence.                 |

| Transition          | Trigger                                | Result                                          |
|---------------------|----------------------------------------|-------------------------------------------------|
| DISCOVERED → ACTIVE | Valid reconciliation                   | Asset becomes normal managed identity.          |
| ACTIVE → STALE      | Freshness threshold                    | Asset remains historically known but not fresh. |
| STALE → ACTIVE      | New valid observation                  | Asset returns to current state.                 |
| STALE → MISSING     | Confirmed absence                      | Asset not currently observed.                   |
| MISSING → RETIRED   | Authoritative retirement/scope removal | Asset leaves active management.                 |
| ANY → CONFLICTED    | Identity ambiguity                     | Unsafe automatic merge blocked.                 |
| CONFLICTED → ACTIVE | Deterministic/manual resolution        | Identity restored.                              |

**ID-LIFE-001:** Identity conflict shall block automatic merging where the result could change security or change-management scope.

## 7. Identifier Resolution and Reconciliation

### 7.1 Resolution Process

32. Parse provider observation into normalized identity attributes.
33. Determine provider/account/scope context.
34. Compute canonical resource key.

35. Look up exact provider identifier match.
36. If unavailable, evaluate approved deterministic secondary keys.
37. Check for collisions/conflicts.
38. Resolve to existing asset or create a new asset.
39. Persist reconciliation evidence.

## 7.2 Reconciliation Outcomes

| Outcome             | Meaning                                                            |
|---------------------|--------------------------------------------------------------------|
| EXACT_MATCH         | Existing asset identified by stable provider identity.             |
| DETERMINISTIC_MATCH | Existing asset identified by approved deterministic composite key. |
| NEW_ASSET           | No matching asset exists.                                          |
| CONFLICT            | Multiple plausible identities exist.                               |
| INVALID             | Required identity attributes are missing/invalid.                  |
| OUT_OF_SCOPE        | Resource exists but is outside current managed scope.              |

**ID-REC-001:** Reconciliation shall record the method used to resolve an observation to an asset.

**ID-REC-002:** A conflict shall not be silently resolved through fuzzy name matching.

## 8. Relationship Model

CloudShield AI models infrastructure relationships as directed, typed edges between assets. A relationship describes a meaningful connection; dependency is a semantic classification of a relationship relevant to operation, security or change impact.

| Relationship Field   | Description                                            |
|----------------------|--------------------------------------------------------|
| edge_id              | Relationship identity.                                 |
| source_asset_id      | Source asset.                                          |
| target_asset_id      | Target asset.                                          |
| relationship_type    | Canonical relationship type.                           |
| direction            | Directed relationship.                                 |
| strength/criticality | Optional relationship significance.                    |
| confidence           | Confidence in relationship derivation.                 |
| valid_from/valid_to  | Temporal validity.                                     |
| source               | Provider API/config/rule/derived deterministic source. |
| evidence_ids         | Supporting evidence.                                   |
| status               | ACTIVE/INVALIDATED/UNKNOWN.                            |

**IDREL-001:** Every safety-relevant relationship shall have a defined source and provenance.

## 9. Dependency Taxonomy

| Category      | Examples                                    | Impact Meaning                      |
|---------------|---------------------------------------------|-------------------------------------|
| Network       | Load balancer →target, subnet →route        | Connectivity/blast radius.          |
| Security      | Security group →instance, IAM role →service | Authorization/exposure impact.      |
| Data          | Application →database, bucket →consumer     | Data availability/integrity impact. |
| Compute       | Service →instance/container                 | Runtime impact.                     |
| Configuration | Resource →policy/configuration              | Behavior/control impact.            |
| Identity      | Principal/role →resource                    | Permission impact.                  |
| Availability  | Service →failover/replica                   | Resilience impact.                  |

|             |                               |                       |
|-------------|-------------------------------|-----------------------|
| Operational | Resource → monitoring/logging | Observability impact. |
|-------------|-------------------------------|-----------------------|

The MVP should implement only relationship types required for the supported AWS resource set and safety-critical workflows. New relationship types must be versioned and documented.

## 10. Dependency Discovery

Dependencies may be discovered from authoritative or deterministic sources in descending preference:

40. Direct cloud-provider relationship APIs.
41. Normalized configuration references.
42. Explicit provider resource attributes.
43. Deterministic rules over known resource relationships.
44. Controlled derived relationships from multiple authoritative observations.

AI may assist with interpretation or investigation but should not be the sole source of a safety-critical dependency edge.

**DEP-DISC-001:** A dependency used to block or authorize a consequential change shall have a deterministic or explicitly reviewed basis.

**DEP-DISC-002:** Dependency discovery shall preserve the source observation/configuration from which an edge was derived.

## 11. Dependency Confidence and Provenance

| Confidence | Functional Interpretation                                          |
|------------|--------------------------------------------------------------------|
| HIGH       | Direct provider relationship or equivalent authoritative evidence. |
| MEDIUM     | Strong deterministic inference from normalized configuration.      |
| LOW        | Derived relationship with meaningful uncertainty.                  |
| UNKNOWN    | Relationship may exist but evidence is insufficient.               |

Confidence is not permission to ignore a relationship. For safety decisions, low/unknown confidence may require additional analysis, human review or a blocked state depending on policy.

**DEP-CONF-001:** The system shall not convert unknown dependency knowledge into a false 'no dependency' result.

## 12. Graph Semantics

### 12.1 Directed Graph

The canonical graph is represented conceptually as  $G = (V, E)$ , where:

- $V$  = CloudShield AI assets.
- $E$  = typed dependency/relationship edges.
- Each edge has source, target, type, provenance and temporal validity.
- Traversal respects tenant, cloud scope, authorization and policy boundaries.

## 12.2 Edge Semantics

| Edge Property | Rule                                                                                      |
|---------------|-------------------------------------------------------------------------------------------|
| Direction     | Source depends on, references or affects target according to relationship type semantics. |
| Type          | Defines traversal meaning.                                                                |
| Validity      | Edge may be current, superseded, invalidated or unknown.                                  |
| Confidence    | Indicates evidence strength.                                                              |
| Scope         | Edge cannot cross unauthorized tenant/scope boundaries.                                   |
| Version       | Changes to relationship interpretation are traceable.                                     |

## 12.3 Cycles

Cycles are valid in infrastructure graphs. Traversal must use visited-node/edge tracking and bounded depth so a cycle cannot cause infinite processing. Cycle presence should be visible when relevant to impact analysis.

## 13. Dependency Queries

| Query               | Purpose                                                  |
|---------------------|----------------------------------------------------------|
| Direct dependencies | Resources directly relied upon by an asset.              |
| Dependents          | Resources that rely on an asset.                         |
| Upstream closure    | Resources recursively required by a subject.             |
| Downstream closure  | Resources potentially affected by a subject.             |
| Path query          | Explain a relationship between two assets.               |
| Impact closure      | Assets potentially affected by a candidate.              |
| Finding propagation | Findings attached to affected dependencies.              |
| Trust propagation   | Trusted assumptions potentially invalidated by a change. |

**DEP-Q-001:** Queries shall support bounded traversal.

**DEP-Q-002:** Queries used in safety decisions shall preserve the graph context/version used for the result.

## 14. Change-Centric Dependency Analysis

The most important dependency operation is analyzing a proposed change. The candidate defines target assets and intended changes; dependency analysis determines direct and transitive effects.

45. Load exact candidate version.
46. Resolve target assets.
47. Load valid dependency graph context.
48. Traverse relevant upstream/downstream relationships.
49. Identify directly affected assets.
50. Identify transitively affected assets.
51. Identify security findings attached to affected assets.
52. Identify trusted-state assumptions dependent on affected resources.
53. Evaluate policy thresholds and safety constraints.
54. Persist the impact/dependency result with graph provenance.

**DEP-CHANGE-001:** Impact analysis shall operate on the exact candidate version being considered.

**DEP-CHANGE-002:** A candidate shall not pass impact analysis when required dependency information is materially incomplete.

## 15. Impact and Blast-Radius Model

| Impact Layer | Definition                                                          |
|--------------|---------------------------------------------------------------------|
| Direct       | Assets explicitly targeted by the candidate.                        |
| First-order  | Assets directly dependent on or required by target assets.          |
| Transitive   | Assets reached through multiple dependency edges.                   |
| Security     | Findings/risk conditions potentially affected.                      |
| Trust        | Trusted states or verification assumptions potentially invalidated. |
| Operational  | Monitoring, logging, availability or operational controls affected. |

### 15.1 Blast-Radius Classification

| Class    | Meaning                                                            |
|----------|--------------------------------------------------------------------|
| LOCAL    | Impact confined to target asset(s).                                |
| BOUNDED  | Impact extends to a known limited dependency set.                  |
| BROAD    | Many connected resources are affected.                             |
| CRITICAL | Critical infrastructure/security/trust boundaries may be affected. |
| UNKNOWN  | Graph information is insufficient for reliable classification.     |

**DEP-IMPACT-001:** UNKNOWN blast radius shall not be treated as LOCAL or BOUNDED.

## 16. Invalidation Semantics

Dependency changes can invalidate previous conclusions. Invalidation does not erase historical results; it marks them as no longer safe to use without re-evaluation.

| Affected Object   | Example Invalidation Trigger                                 |
|-------------------|--------------------------------------------------------------|
| Finding           | Resource relationship changes security exposure.             |
| Risk Assessment   | Critical dependency or exposure context changes.             |
| Impact Analysis   | Candidate or graph context changes.                          |
| Verification      | Dependency changed after verification.                       |
| Trusted State     | Trusted assumption depends on a materially changed resource. |
| AI Recommendation | Context used by recommendation materially changed.           |
| Approval          | Protected impact context changed after approval.             |

**DEP-INV-001:** Invalidation shall identify the cause and affected object.

**DEP-INV-002:** Invalidation shall trigger re-analysis or explicit review where policy requires it.

## 17. DAIL Integration

DAIL consumes CloudShield AI identity and dependency semantics to make infrastructure-change lifecycle decisions deterministic.

| DAIL Function         | Identity/Dependency Input                            |
|-----------------------|------------------------------------------------------|
| Candidate identity    | Exact candidate/version and target asset identities. |
| Dependency assessment | Current valid graph and relationship provenance.     |
| Impact analysis       | Direct/transitive dependency closure.                |
| Invalidation          | Relationships affected by proposed change.           |

|              |                                                                 |
|--------------|-----------------------------------------------------------------|
| Verification | Expected dependencies and post-change relationships.            |
| Promotion    | Confirmation that required dependency assumptions remain valid. |

**DAIL-ID-001:** DAIL shall use stable CloudShield AI asset identities for lifecycle references.

**DAIL-DEP-001:** DAIL shall not rely on mutable display names to identify change targets.

**DAIL-DEP-002:** DAIL impact gates shall distinguish incomplete dependency knowledge from a clean result.

## 18. Consistency, Freshness and Versioning

| Property         | Requirement                                                                     |
|------------------|---------------------------------------------------------------------------------|
| Freshness        | Edges used for safety decisions must satisfy configured freshness requirements. |
| Version          | Graph/edge context used in analysis is identifiable.                            |
| Snapshot         | Impact results preserve the dependency context used.                            |
| Recalculation    | Material relationship changes trigger relevant re-analysis.                     |
| Historical truth | Older graph states remain available where needed for audit.                     |
| Provider lag     | Provider observation delays are represented as freshness uncertainty.           |

**DEP-FRESH-001:** A stale dependency graph shall not silently satisfy a freshness-sensitive safety gate.

**DEP-FRESH-002:** Dependency analysis results shall record the graph context/version used.

## 19. Concurrency and Reconciliation

- Two discovery workers may observe the same asset; reconciliation must be idempotent.
- A relationship may be discovered concurrently from multiple sources; duplicate equivalent edges should reconcile.
- Conflicting relationship evidence should produce explicit conflict/uncertainty rather than silent replacement.
- A dependency update occurring during candidate verification may invalidate the verification context.
- Promotion should re-check critical dependency assumptions immediately before trusted-state advancement.

**DEP-CONC-001:** Concurrent dependency updates shall not cause a stale analysis to overwrite a newer authoritative relationship state.

**DEP-CONC-002:** Relationship reconciliation shall preserve competing evidence when conflicts cannot be deterministically resolved.

## 20. Security and Authorization

- Graph queries are tenant-scoped.
- Resource-level authorization applies to asset and edge visibility.
- Sensitive relationships may have additional access classification.
- AI context assembly may only include dependencies the requesting actor/service is authorized to access.
- Audit views must not expose unauthorized graph information through relationship metadata.
- Provider credentials and secrets are never dependency payloads.

**DEP-AUTH-001:** Unauthorized assets must not become discoverable through graph traversal from an authorized asset.

**DEP-AUTH-002:** Dependency edges crossing an authorization boundary shall be masked or represented only according to policy.

## 21. Persistence Model

The MVP should store canonical assets and dependency edges in the relational system of record. This avoids introducing a dedicated graph database before graph query scale requires it.

| Data                        | MVP Storage                          | Purpose                                           |
|-----------------------------|--------------------------------------|---------------------------------------------------|
| Asset                       | Relational                           | Canonical identity and current metadata.          |
| ResourceIdentifier          | Relational                           | Provider/native identity mapping.                 |
| DependencyEdge              | Relational                           | Typed relationship and provenance.                |
| Observation                 | Relational + object storage          | Source evidence.                                  |
| ImpactAnalysis              | Relational                           | Candidate-specific graph result.                  |
| EvidenceArtifact            | Relational metadata + object storage | Large supporting artifacts.                       |
| Graph cache/materialization | Optional                             | Performance optimization only; not authoritative. |

If a graph database is introduced later, it should be treated as a query/indexing representation of the canonical domain rather than a second conflicting source of truth.

## 22. MVP Dependency Coverage

MVP should prioritize relationship types that prove the end-to-end safety loop.

| Priority | Coverage                                                                     |
|----------|------------------------------------------------------------------------------|
| P0       | Asset identity and provider resource mapping.                                |
| P0       | Network relationships required for supported AWS resources.                  |
| P0       | Security-group/IAM/policy relationships required by selected rules.          |
| P0       | Application-to-data/resource relationships needed for supported remediation. |
| P0       | Direct and bounded transitive traversal.                                     |
| P0       | Impact and invalidation integration with candidates.                         |
| P0       | Evidence/provenance for safety-relevant edges.                               |
| P1       | Advanced topology and broad AWS service coverage.                            |
| P1       | Graph-optimized persistence.                                                 |
| P2       | Cross-cloud relationship normalization beyond MVP.                           |

## 23. Example Dependency Scenarios

### Scenario A — Security Group Change

55. Candidate targets a security group.
56. Graph identifies attached workloads.
57. Impact analysis marks those workloads as directly/transitively affected.

- 58. Security findings associated with affected exposure are identified.
- 59. Verification checks expected security posture.
- 60. Promotion occurs only if technical and governance gates pass.

### Scenario B — Database Configuration Change

- 61. Candidate targets a database configuration.
- 62. Graph identifies application consumers.
- 63. Impact analysis determines application blast radius.
- 64. Relevant availability/security findings are surfaced.
- 65. Approval policy evaluates impact.
- 66. Verification confirms intended database and dependent behavior.

### Scenario C — IAM Role Change

- 67. Candidate modifies an IAM role/policy.
- 68. Graph resolves principals/resources using that identity.
- 69. Affected access paths are identified.
- 70. Security/risk analysis is refreshed.
- 71. Verification evaluates intended permissions and security invariants.
- 72. Promotion is blocked if required evidence is unavailable.

### Scenario D — Dependency Data Incomplete

- 73. Candidate targets a resource with incomplete dependency coverage.
- 74. Impact engine marks analysis INCOMPLETE/UNKNOWN.
- 75. Policy evaluates whether uncertainty is acceptable.
- 76. If not acceptable, candidate is BLOCKED or ESCALATED.
- 77. No false clean-impact result is produced.

## 24. Acceptance Criteria

**IDDEP-ACC-01:** Cloud resources can be represented by stable CloudShield AI asset identities.

**IDDEP-ACC-02:** Provider identifiers remain traceable to canonical assets.

**IDDEP-ACC-03:** Mutable display names cannot become sole identity keys.

**IDDEP-ACC-04:** Identity conflicts are explicit and do not silently merge resources.

**IDDEP-ACC-05:** Assets and relationships are tenant and scope isolated.

**IDDEP-ACC-06:** Dependency edges are typed and directed where semantics require it.

**IDDEP-ACC-07:** Safety-relevant relationships have provenance.

**IDDEP-ACC-08:** Dependency confidence/uncertainty is represented.

**IDDEP-ACC-09:** Graph traversal is bounded and cycle-safe.

**IDDEP-ACC-10:** Dependency queries preserve graph context/version for consequential analysis.

**IDDEP-ACC-11:** Change-centric impact analysis operates on the exact candidate version.

**IDDEP-ACC-12:** Direct and transitive impact are distinguishable.

**IDDEP-ACC-13:** Unknown/incomplete dependency information cannot be treated as a clean result.

**IDDEP-ACC-14:** Material dependency changes can invalidate findings, risk, verification, approval or trusted assumptions.

**IDDEP-ACC-15:** DAIL can consume stable asset identities and dependency/impact results.

**IDDEP-ACC-16:** Promotion re-checks critical dependency assumptions.

**IDDEP-ACC-17:** Concurrent relationship discovery is idempotent.

**IDDEP-ACC-18:** Conflicting relationship evidence remains traceable.

**IDDEP-ACC-19:** The MVP can implement the canonical graph relationally.

**IDDEP-ACC-20:** A future graph database can be introduced without changing canonical domain semantics.

## 25. Traceability

| Capability                 | Domain/Data Alignment             | Lifecycle/Functional Alignment |
|----------------------------|-----------------------------------|--------------------------------|
| Resource identity          | Asset/ResourceIdentifier          | Asset lifecycle                |
| Observation reconciliation | Observation/Asset                 | Discovery lifecycle            |
| Dependency graph           | DependencyEdge                    | Dependency lifecycle           |
| Impact                     | ImpactAnalysis                    | Candidate lifecycle / DAIL     |
| Invalidation               | InvalidationRecord                | Trusted/verification lifecycle |
| AI context                 | AIContext + dependency references | AI lifecycle                   |
| Verification               | Dependency expectations           | Verification lifecycle         |
| Promotion                  | TrustedState/Promotion            | Promotion lifecycle            |
| UX                         | Asset/dependency views            | Inventory and impact UX        |

This document is the authoritative identity and dependency semantic layer for subsequent security, AI, DAIL, AWS and API specifications.

## 26. Glossary

| Term                   | Definition                                                             |
|------------------------|------------------------------------------------------------------------|
| Asset                  | Canonical CloudShield AI representation of an infrastructure resource. |
| Provider Resource ID   | Cloud-provider identifier for a resource.                              |
| Canonical Resource Key | Deterministic normalized identity key.                                 |
| Dependency             | Typed relationship relevant to operation, security or change impact.   |
| Dependency Edge        | Persisted directed relationship between assets.                        |
| Graph                  | Set of assets and relationships used for dependency analysis.          |
| Traversal              | Process of following graph relationships.                              |
| Impact Analysis        | Evaluation of resources and assumptions affected by a candidate.       |
| Blast Radius           | Scope of direct/transitive consequences of a change.                   |
| Invalidation           | Marking a previous result/assumption no longer valid.                  |
| Provenance             | Information describing the source and evidence behind a result.        |
| Confidence             | Strength of evidence supporting a relationship.                        |
| DAIL                   | Deterministic AI Infrastructure Lifecycle subsystem.                   |



## Document Completion Statement

CloudShield AI Identity & Dependency Specification v1.0 establishes the stable identity and relationship semantics required for safe infrastructure intelligence and change management. It ensures that dependency analysis is evidence-backed, scope-aware, deterministic where safety requires it, and integrated with DAIL without turning DAIL into the entire product.

Next planned document: 08 — CloudShield AI Security & Threat Model Specification.